

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT - CO2003

BÀI TẬP LỚN 1

BKMove: QUẢN LÝ TUYẾN XE BUÝT VÀ TÌM KIẾM HÀNH TRÌNH

Thành phố Hồ Chí Minh, 09/2026

ĐẶC TẢ BÀI TẬP LỚN

Phiên bản 1.0

1 Kết quả đạt được

Sau khi hoàn thành bài tập lớn này, sinh viên có khả năng:

- Hiện thực và thao tác trên các cấu trúc danh sách liên kết tuyến tính và danh sách liên kết vòng.
- Áp dụng cấu trúc dữ liệu trừu tượng để mô hình hóa một ứng dụng thực tế nhỏ.
- Hiện thực thuật toán Quick Sort tổng quát và sử dụng comparator tùy biến để xác định thứ tự riêng của ứng dụng.
- Kết hợp cấu trúc dữ liệu và giải thuật để giải quyết bài toán tìm tuyến trực tiếp và tìm hành trình có tối đa một lần chuyển tuyến.

2 Giới thiệu

Các ứng dụng giao thông công cộng phải trả lời nhiều truy vấn có vẻ đơn giản đối với người dùng nhưng lại đòi hỏi cách tổ chức dữ liệu cẩn thận ở bên trong. Một tuyến xe buýt có một dãy trạm theo chiều đi và một dãy trạm theo chiều về. Hai dãy này không nhất thiết là đảo ngược chính xác của nhau. Một trạm có thể xuất hiện ở cả hai chiều, trong khi một số trạm chỉ xuất hiện ở một chiều.

Trong bài tập lớn này, sinh viên sẽ xây dựng một ứng dụng quản lý tuyến xe buýt đơn giản có tên **BKMove**. Bài tập lớn bắt đầu từ việc hiện thực các cấu trúc danh sách liên kết cơ bản, sau đó từng bước tích hợp chúng vào hệ thống quản lý tuyến và tìm kiếm hành trình. Hệ thống không mô hình hóa toàn bộ mạng giao thông của thành phố dưới dạng đồ thị. Thay vào đó, bài tập lớn tập trung vào các hành trình đi trực tiếp hoặc có tối đa một lần chuyển tuyến.

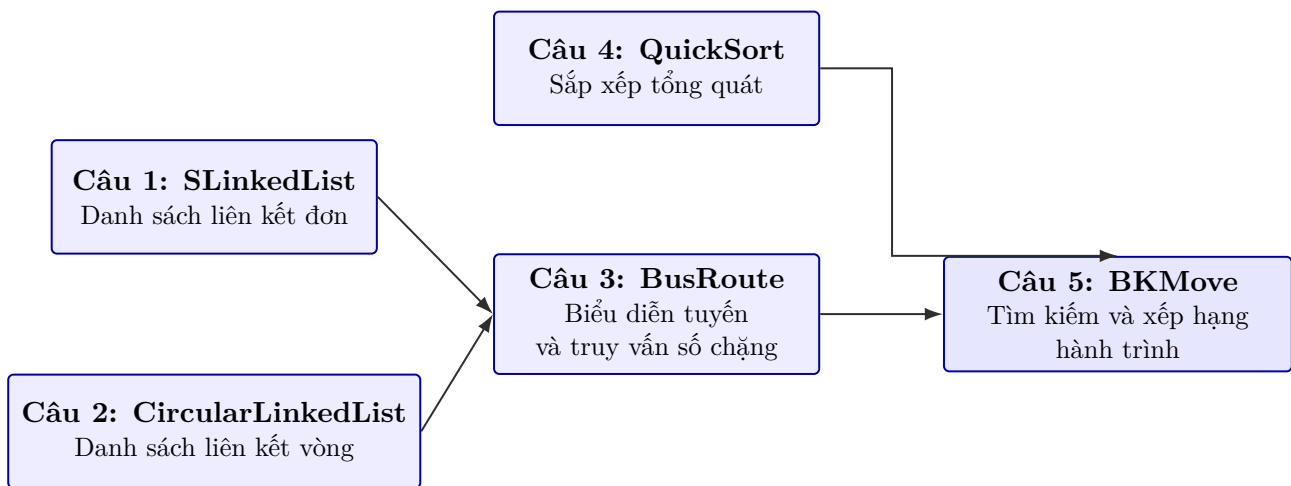
bài tập lớn gồm năm câu:

- **Câu 1 – Singly Linked List:** Hiện thực các thao tác được yêu cầu của `SLinkedList<T>`.
- **Câu 2 – Circular Linked List:** Hiện thực `CircularLinkedList<T>` và duy trì đúng cấu trúc vòng.

- **Câu 3 – Bus Route:** Xây dựng lớp trừu tượng biểu diễn một tuyến xe buýt dựa trên các cấu trúc danh sách liên kết và hỗ trợ các truy vấn phụ thuộc chiều di chuyển.
- **Câu 4 – Quick Sort:** Hiện thực Quick Sort tổng quát với comparator và cơ chế chọn pivot tùy biến.
- **Câu 5 – BKMove Search:** Tìm kiếm và xếp hạng các hành trình trực tiếp và các hành trình có tối đa một lần chuyển tuyến.

Mỗi câu có **2 điểm**, tổng cộng **10 điểm**.

Các câu được thiết kế để hoàn thành theo từng bước. Câu 1 và Câu 2 cung cấp nền tảng cấu trúc dữ liệu cho Câu 3. Câu 4 cung cấp thuật toán sắp xếp được sử dụng ở Câu 5. Cuối cùng, Câu 5 kết hợp các thành phần trước đó thành ứng dụng BKMove hoàn chỉnh.



Hình 1: Luồng phụ thuộc giữa năm câu trong bài tập BKMove.

3 Kiến thức nền và mô hình ứng dụng

3.1 Singly Linked List

Singly linked list lưu các phần tử trong những node được nối với nhau bằng con trỏ **next**. Khác với mảng, các node không cần nằm liên tiếp trong bộ nhớ. Vì vậy, việc chèn và xóa phần tử phụ thuộc vào thao tác trên các liên kết giữa các node thay vì dịch chuyển một vùng phần tử liên tiếp.

Starter code cung cấp định nghĩa node, metadata của danh sách, constructor, destructor, iterator và một số phương thức tiện ích. Sinh viên chịu trách nhiệm hiện thực các thao tác cốt lõi được yêu cầu bởi interface `IList<T>`.

3.2 Circular Linked List

Circular linked list là danh sách liên kết trong đó node cuối cùng liên kết trở lại node đầu tiên thay vì kết thúc bằng `nullptr`. Vì vậy, cách duyệt đúng đối với singly linked list thông thường chưa chắc đúng đối với circular linked list.

Cách hiện thực trong bài tập lớn này sử dụng các trường `head`, `tail` và `count`. Sinh viên phải đảm bảo cấu trúc vòng luôn hợp lệ sau mọi thao tác chèn, xóa và xóa toàn bộ danh sách.

3.3 Tuyến xe buýt và chiều di chuyển

Mỗi tuyến xe buýt có hai chiều logic:

- OUTBOUND: đi từ bến đầu thứ nhất đến bến đầu thứ hai.
- INBOUND: đi từ bến đầu thứ hai trở về bến đầu thứ nhất.

Dãy trạm theo chiều đi và chiều về không được giả định là ảnh đảo ngược chính xác của nhau. Ví dụ, hai dãy sau là hợp lệ:

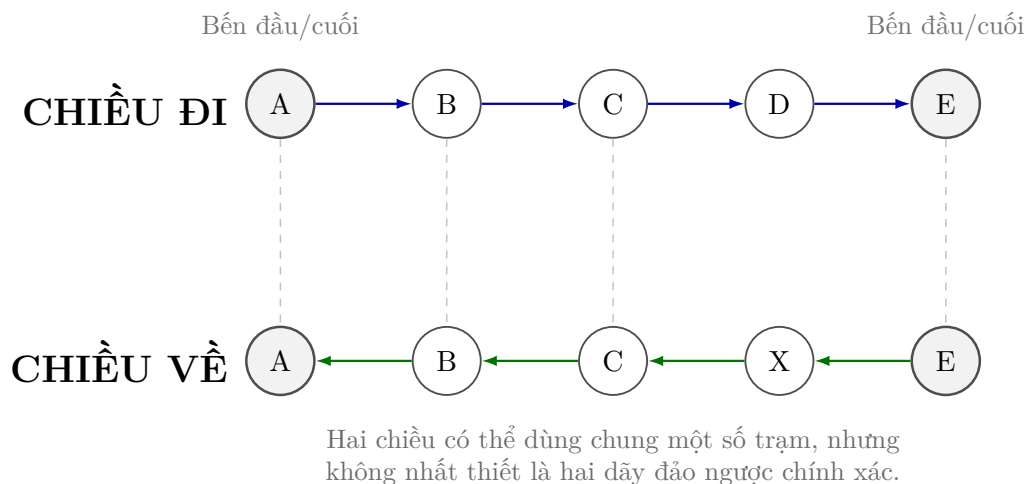
```
OUTBOUND: A -> B -> C -> D -> E
INBOUND: E -> X -> C -> B -> A
```

Một `BusRoute` lưu một vòng hành trình hoàn chỉnh bằng `CircularLinkedList<BusStop>` và phải cung cấp cả hai chiều thông qua public API. Cấu trúc lưu trữ bên trong đã được cố định bởi starter code; sinh viên cần xác định cách ánh xạ đúng các cách nhìn logic theo từng chiều lên cấu trúc lưu trữ đó.

3.4 Quick Sort và xếp hạng

Ứng dụng `BKMove` có thể tạo ra nhiều tuyến hoặc hành trình hợp lệ cho cùng một truy vấn. Các ứng viên này phải được trả về theo một thứ tự xếp hạng xác định. Trước tiên, sinh viên sẽ hiện thực **Quick Sort** tổng quát; sau đó tái sử dụng thuật toán này ở Câu 5 với các comparator riêng của ứng dụng.

Quick Sort là thuật toán sắp xếp theo chiến lược chia để trị. Thuật toán liên tục chọn một pivot, partition miền hiện tại quanh pivot, rồi đệ quy sắp xếp các miền con. Starter code cung cấp một mức phân rã bài toán thông qua `sort`, `quickSort` và `partition`; sinh viên có thể tự bổ sung các private helper nếu thấy cần thiết.



Hình 2: Chiều đi và chiều về của một tuyến xe buýt.

4 Yêu cầu

Sinh viên phải hoàn thành năm câu dưới đây. Không được thay đổi public API trong các header được cung cấp. Public tests đi kèm starter package và được xem là một phần của đặc tả có thể thực thi, đặc biệt đối với các trường hợp biên và hành vi đặc biệt.

4.1 Câu 1 – Lớp SLinkedList<T> (2 điểm)

Lớp SLinkedList<T> hiện thực interface IList<T> đã được cung cấp. Lớp sử dụng cấu trúc liên kết dựa trên con trỏ và lưu số phần tử hiện tại trong `count`.

Cấu trúc lớp liên quan được trình bày dưới đây. Các phương thức đã được cung cấp được lược bỏ để ngắn gọn.

```

1 template<class T>
2 class SLinkedList : public IList<T> {
3 public:
4     class Iterator;
5     class Node;
6
7 protected:
8     Node* head;
9     Node* tail;
10    int count;
11    bool (*itemEqual)(T& lhs, T& rhs);
12    void (*deleteUserData)(SLinkedList<T>*);
13

```

```
14 public:
15     void add(T e) override;
16     void add(int index, T e) override;
17     T removeAt(int index) override;
18     bool removeItem(T item,
19                     void (*removeItemData)(T) = 0) override;
20     void clear() override;
21     T& get(int index) override;
22     int indexOf(T item) override;
23
24     // Other methods are provided in the starter code.
25 };
```

Sinh viên phải hiện thực các phương thức sau:

1. **void add(T e);**

- Thêm *e* vào cuối danh sách.
- Thao tác append này bắt buộc chạy trong thời gian $O(1)$.

2. **void add(int index, T e);**

- Chèn *e* vào vị trí được yêu cầu.
- Hành vi tại biên hợp lệ và không hợp lệ được thể hiện trong public tests.

3. **T removeAt(int index);**

- Xóa phần tử tại vị trí được yêu cầu và trả về giá trị bị xóa.
- Metadata và các liên kết của danh sách phải vẫn đúng sau khi xóa phần tử đầu, cuối, giữa hoặc phần tử duy nhất.

4. **bool removeItem(T item, void (*removeItemData)(T) = 0);**

- Xóa phần tử khớp đầu tiên theo quy tắc so sánh bằng được lớp sử dụng.
- Nếu *removeItemData* được cung cấp, callback này phải được áp dụng lên giá trị bị xóa theo hành vi thể hiện trong public tests.

5. **void clear();**

- Xóa toàn bộ data node và đưa danh sách về trạng thái rỗng có thể tái sử dụng.
- Bộ nhớ node được cấp phát động phải được giải phóng đúng cách.

6. **T& get(int index);**

- Trả về tham chiếu đến giá trị tại vị trí được yêu cầu.

7. **int indexOf(T item);**

- Trả về vị trí của phần tử khớp đầu tiên, hoặc giá trị biểu diễn không tìm thấy như trong public tests.
- Việc so khớp phải sử dụng callback `itemEqual` nếu callback này được cung cấp.

Sinh viên không được thay thế cấu trúc linked list bắt buộc bằng STL sequence container.

4.2 Câu 2 – Lớp `CircularLinkedList<T>` (2 điểm)

Lớp `CircularLinkedList<T>` cũng hiện thực `IList<T>`, nhưng lưu các phần tử trong một cấu trúc node dạng vòng.

```
1 template<class T>
2 class CircularLinkedList : public IList<T> {
3 public:
4     class Node;
5
6 protected:
7     Node* head;
8     Node* tail;
9     int count;
10    bool (*itemEqual)(T& lhs, T& rhs);
11    void (*deleteUserData)(CircularLinkedList<T>*);
12
13 public:
14    void add(T e) override;
15    void add(int index, T e) override;
16    T removeAt(int index) override;
17    bool removeItem(T item,
18                    void (*removeItemData)(T) = 0) override;
19    void clear() override;
20    T& get(int index) override;
21    int indexOf(T item) override;
22
23    // Other methods are provided in the starter code.
24 };
```

Sinh viên phải hiện thực cùng bảy thao tác cốt lõi như ở Câu 1, nhưng mọi thao tác phải duy trì đúng cấu trúc vòng. Đặc biệt, hiện thực phải xử lý đúng các trạng thái chuyển tiếp giữa danh sách rỗng, danh sách một node và danh sách có nhiều node.

Thao tác append `add(T e)` phải chạy trong thời gian $O(1)$. Sinh viên không được thay thế circular linked list bắt buộc bằng STL sequence container.

Public tests kiểm tra thao tác chèn, xóa, chỉ số không hợp lệ, equality callback, hành vi copy, clear và khả năng tái sử dụng danh sách sau khi danh sách trở về rỗng.

4.3 Câu 3 – Lớp BusRoute (2 điểm)

Lớp BusRoute biểu diễn một vòng phục vụ hoàn chỉnh của một tuyến xe buýt. Public interface và các data member bên trong được cố định bởi starter code.

```
1 class BusRoute {
2 private:
3     string routeId;
4     CircularLinkedList<BusStop> stops;
5     int outboundCount;
6     bool built;
7
8 private:
9     int physicalIndex(int index, Direction direction);
10
11 public:
12     explicit BusRoute(string routeId);
13
14     string getId() const;
15     bool isBuilt() const;
16
17     void build(SLinkedList<BusStop>& outbound,
18               SLinkedList<BusStop>& inbound);
19     int getStopCount(Direction direction);
20     BusStop& getStop(int index, Direction direction);
21     int getHopCount(string fromStopId,
22                     string toStopId,
23                     Direction direction);
24 };
```

Input của build được đảm bảo hợp lệ. Mỗi danh sách theo một chiều có ít nhất hai trạm. Trạm cuối của outbound là trạm đầu của inbound, và trạm cuối của inbound là trạm đầu của outbound. Trong cùng một chiều, các stop ID là duy nhất. Một trạm vẫn có thể xuất hiện ở cả hai chiều vì hai chiều tương ứng với hai lượt ghé khác nhau của tuyến.

Sinh viên phải hiện thực các phương thức sau:

1. **void build(SLinkedList<BusStop>& outbound, SLinkedList<BusStop>& inbound);**
 - Xây dựng tuyến từ hai dãy trạm logic theo hai chiều.

- Hai danh sách input phải giữ nguyên, không được thay đổi.
- Gọi build lần nữa sẽ thay thế hoàn toàn tuyến đã được xây dựng trước đó.

2. **int getStopCount(Direction direction);**

- Trả về số trạm nhìn thấy trong chiều logic được chọn.

3. **int physicalIndex(int index, Direction direction);**

- Ánh xạ một logical index theo chiều sang vị trí tương ứng trong cấu trúc circular storage bên trong.
- Đây là private method và được cung cấp như một mức scaffolding cho abstraction của tuyến.

4. **BusStop& getStop(int index, Direction direction);**

- Trả về trạm tại logical position được yêu cầu trong chiều được chọn.
- Hành vi tại biên được quy định bởi public tests.

5. **int getHopCount(string fromStopId, string toStopId, Direction direction);**

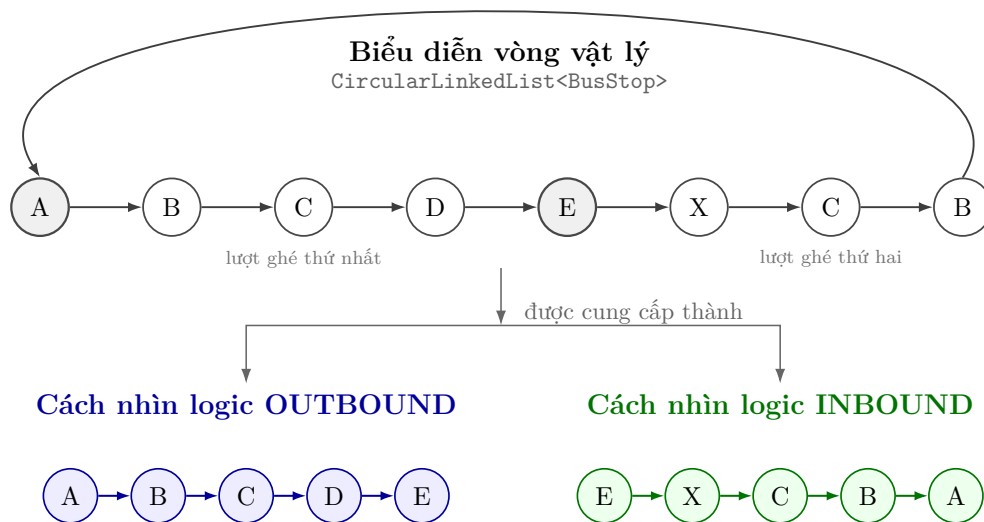
- Trả về số đoạn tuyến cần đi từ fromStopId đến toStopId, đồng thời tuân thủ chiều và thứ tự trạm đã chọn.
- Nếu hành trình theo chiều yêu cầu không hợp lệ, phương thức trả về giá trị thất bại được quy định trong public tests.

Hiện thực của BusRoute bắt buộc sử dụng CircularLinkedList<BusStop> như representation đã yêu cầu. Không được dùng auxiliary STL sequence container trong BusRoute.cpp.

4.4 Câu 4 – Lớp QuickSort<T> (2 điểm)

Lớp QuickSort<T> hiện thực interface ISort<T> đã được cung cấp.

```
1 template<class T>
2 class QuickSort : public ISort<T> {
3 private:
4     int (*pivotSelection)(T*, int);
5
6 public:
7     QuickSort(int (*pivotSelection)(T*, int) = 0);
8
9     void sort(T array[], int size,
10              int (*comparator)(T&, T&) = 0) override;
11
```



Hình 3: Một biểu diễn vòng duy nhất của tuyến xe buýt và hai cách nhìn logic theo chiều di chuyển.

```

12 private:
13     void quickSort(T array[], int left, int right,
14                     int (*comparator)(T&, T&) = 0);
15
16     int partition(T array[], int left, int right,
17                   int (*comparator)(T&, T&) = 0);
18 };

```

Sinh viên phải hiện thực cả ba phương thức được thể hiện ở trên. Cách phân rã được cung cấp cố ý chỉ dừng ở một mức; sinh viên có thể tự thêm private helper nếu thấy hữu ích.

- Khi không có comparator tùy biến, lớp phải sắp xếp các phần tử theo thứ tự tăng dần bằng các quan hệ < và > thông thường.
- Khi có comparator, kết quả phải tuân theo quy ước thứ tự được thể hiện trong public tests.
- Nếu có callback `pivotSelection` tùy biến, hiện thực phải sử dụng callback này khi chọn pivot.
- Hiện thực phải xử lý đúng mảng rỗng, mảng một phần tử, giá trị trùng nhau, input đã được sắp xếp, input sắp xếp ngược và input gồm toàn các giá trị bằng nhau.
- Không được sử dụng các hàm sắp xếp chuẩn như `std::sort`, `std::stable_sort` và `qsort`.

4.5 Câu 5 – Lớp BKMove (2 điểm)

Lớp BKMove lưu một tập các tuyến xe buýt và cung cấp chức năng tìm kiếm cuối cùng của bài tập lớn.

```
1 class BKMove {
2 private:
3     vector<BusRoute*> routes;
4
5 public:
6     BKMove() = default;
7
8     void addRoute(BusRoute* route);
9     int getRouteCount() const;
10    BusRoute* getRoute(int index);
11
12    vector<RouteResult> findDirectRoutes(
13        string fromStopId,
14        string toStopId
15    );
16
17    vector<JourneyResult> findJourneys(
18        string fromStopId,
19        string toStopId
20    );
21 };
```

Starter code đã hiện thực `addRoute`, `getRouteCount` và `getRoute`. Sinh viên phải hiện thực hai phương thức tìm kiếm.

4.5.1 Tìm tuyến trực tiếp

Phương thức

```
vector<RouteResult> findDirectRoutes(string fromStopId, string toStopId)
```

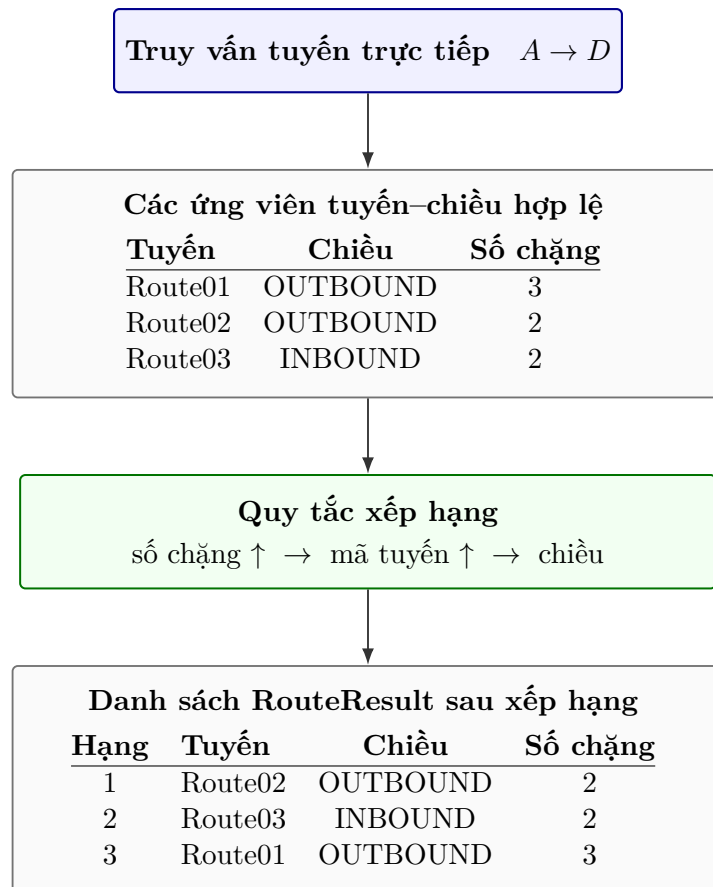
phải tìm mọi cặp tuyến–chiều có thể đi từ trạm xuất phát đến trạm đích mà không cần chuyển tuyến. Nếu cùng một tuyến hỗ trợ cả hai chiều thì hai chiều được xem là hai ứng viên riêng biệt.

Các kết quả trả về phải được xếp hạng theo thứ tự ưu tiên sau:

1. `hopCount` nhỏ hơn đứng trước.

2. Nếu bằng nhau, `routeId` nhỏ hơn theo thứ tự từ điển đứng trước.
3. Nếu vẫn bằng nhau, `OUTBOUND` đứng trước `INBOUND`.

Việc xếp hạng bắt buộc sử dụng hiện thực QuickSort từ Câu 4.



Hình 4: Tìm và xếp hạng các tuyến trực tiếp cho truy vấn từ trạm A đến trạm D.

4.5.2 Tìm hành trình với tối đa một lần chuyển tuyến

Phương thức

```
vector<JourneyResult> findJourneys(string fromStopId, string toStopId)
```

phải trả về toàn bộ hành trình hợp lệ sử dụng không lần chuyển tuyến hoặc đúng một lần chuyển tuyến.

Một hành trình có một lần chuyển tuyến gồm hai `routeId` khác nhau. Trạm chuyển tuyến phải xuất hiện sau trạm xuất phát trên chặng thứ nhất và trước trạm đích trên chặng thứ hai; do đó mỗi chặng phải có ít nhất một hop. Nếu có nhiều trạm chuyển tuyến khác nhau tạo

thành các hành trình hợp lệ, mỗi lựa chọn hợp lệ được biểu diễn bằng một `JourneyResult` riêng.

Đối với hành trình có một lần chuyển tuyến, `totalHops` là tổng số hop của hai chặng.

Các hành trình trả về phải được xếp hạng theo thứ tự ưu tiên sau:

1. Ít lần chuyển tuyến hơn đứng trước.
2. `totalHops` nhỏ hơn.
3. `firstRouteId` nhỏ hơn theo thứ tự từ điển.
4. OUTBOUND đứng trước INBOUND đối với `firstDirection`.
5. `transferStopId` nhỏ hơn theo thứ tự từ điển.
6. `secondRouteId` nhỏ hơn theo thứ tự từ điển.
7. OUTBOUND đứng trước INBOUND đối với `secondDirection`.

Các hành trình trực tiếp phải được đưa vào cùng result vector và vì thế đứng trước mọi hành trình có một lần chuyển tuyến theo tiêu chí xếp hạng thứ nhất. Nếu không tồn tại hành trình hợp lệ, phương thức trả về vector rỗng.

Sinh viên được phép dùng `std::vector` cho tập route của `BKMove` và các result collection. Tuy nhiên, không được sao chép topology của tuyến sang một auxiliary STL container chứa `BusStop`. Logic tìm kiếm phải sử dụng public abstraction của `BusRoute`. Không được sử dụng các hàm sắp xếp chuẩn; việc xếp hạng phải dùng `QuickSort` từ Câu 4.

Câu này được giới hạn có chủ đích ở tối đa một lần chuyển tuyến. Không yêu cầu general graph search trong bài tập lớn này.

5 Public tests và các mốc kiểm tra

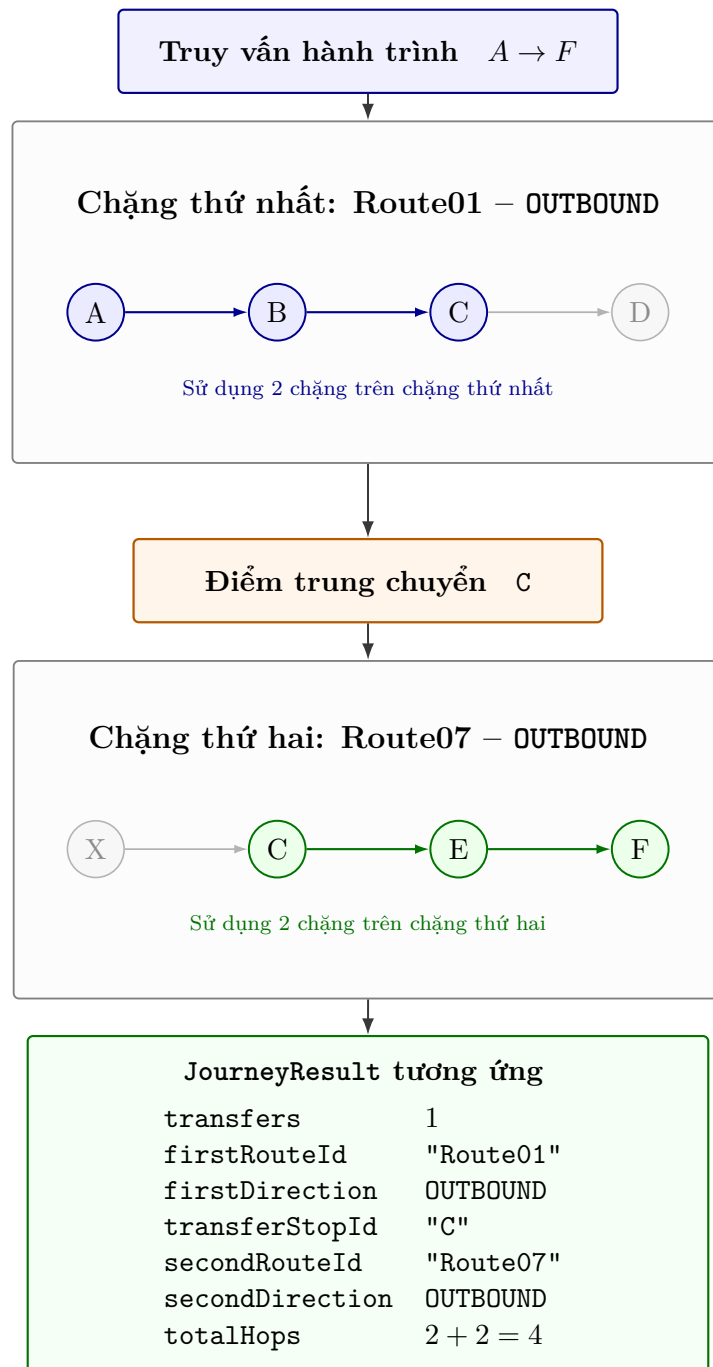
Starter package chứa các public test được nhóm theo từng câu. Sinh viên nên sử dụng các test này theo từng bước trong quá trình hiện thực bài tập lớn.

Chạy toàn bộ public tests của một câu bằng:

```
make q1, make q2, make q3, make q4, hoặc make q5
```

Chạy toàn bộ public tests bằng:

```
make all
```



Hình 5: Ví dụ về một hành trình có một lần trung chuyển và JourneyResult tương ứng.

Mỗi câu có nhiều test group có tên riêng. Để chạy một group trong lúc debug, sử dụng:

```
make q3 TEST=4
```

Các nhóm public test được tóm tắt như sau:

- **Q1 – SLinkedList:** 7 group kiểm tra trạng thái rỗng/một phần tử, chèn, chỉ số, xóa,

equality/callback, copy/assignment và clear/reuse.

- **Q2 – CircularLinkedList:** 7 group kiểm tra các thao tác tương tự trong ngữ cảnh circular list.
- **Q3 – BusRoute:** 7 group kiểm tra trạng thái ban đầu, build, directional view, boundary của chỉ số, hop count, tuyến tối thiểu và rebuild.
- **Q4 – QuickSort:** 5 group kiểm tra base case, sắp xếp mặc định, thứ tự input khó, comparator tùy biến và pivot selection tùy biến.
- **Q5 – BKMove:** 10 group kiểm tra direct search, direct ranking, one-transfer discovery, tính hợp lệ của transfer, toàn bộ ranking tie-break và trường hợp không có kết quả.

Public tests cố ý không bao phủ toàn bộ trường hợp. Hidden tests có thể sử dụng thêm dữ liệu và chuỗi thao tác khác nhưng vẫn tuân theo cùng contract. Các yêu cầu về cấu trúc và độ phức tạp cũng có thể được kiểm tra riêng.

6 Hướng dẫn

Để hoàn thành bài tập lớn này, sinh viên phải:

1. Đọc đặc tả này và xem starter code cùng public tests trước khi bắt đầu hiện thực các câu.
2. Tải và giải nén starter package. Các file quan trọng gồm:
 - `IList.h`, `ISort.h`, `main.h`: các interface và kiểu dữ liệu dùng chung đã được cung cấp.
 - `SLinkedList.h`: Câu 1.
 - `CircularLinkedList.h`: Câu 2.
 - `BusRoute.h`, `BusRoute.cpp`: Câu 3. Header được cố định; sinh viên hiện thực các phương thức được yêu cầu trong `BusRoute.cpp`.
 - `QuickSort.h`: Câu 4.
 - `BKMove.h`, `BKMove.cpp`: Câu 5. Header được cố định; sinh viên hiện thực các phương thức được yêu cầu trong `BKMove.cpp`.
 - `tests/`: public tests.
 - `Makefile`: compile, các checkpoint công khai và các source-rule check cơ bản.
3. Sinh viên không được thay đổi public API trong `IList.h`, `ISort.h`, `BusRoute.h`, `BKMove.h` hoặc `main.h`. Hệ thống chấm có thể thay các file này bằng bản gốc.

4. Sinh viên được phép bổ sung private helper method hoặc internal class khi cần, miễn là public API và representation bắt buộc vẫn được giữ nguyên.
5. Áp dụng các giới hạn source code sau:
 - Không được dùng các STL sequence container như `vector`, `list`, `forward_list` và `deque` để hiện thực `SLinkedList`, `CircularLinkedList` hoặc `BusRoute`.
 - Không được dùng standard sorting function cho Câu 4 hoặc Câu 5.
 - `BKMove` được phép dùng `std::vector` để lưu route pointer và result object, nhưng không được copy các stop của route sang auxiliary STL container chứa `BusStop`.
 - Câu 5 bắt buộc sử dụng hiện thực `QuickSort` từ Câu 4 để xếp hạng.
6. Sinh viên phải giải phóng đúng bộ nhớ node được cấp phát động. Hidden grading có thể gồm các chuỗi thao tác dài và các kiểm tra liên quan đến bộ nhớ.
7. Chương trình sẽ được chấm trong môi trường UNIX bằng compiler C++17. Sinh viên cần tự xử lý mọi khác biệt về compile hoặc runtime trước khi nộp bài chính thức.

7 Nộp bài

Sinh viên chỉ nộp năm file sau:

- `SLinkedList.h`
- `CircularLinkedList.h`
- `BusRoute.cpp`
- `QuickSort.h`
- `BKMove.cpp`

Deadline và đường dẫn nộp bài sẽ được thông báo trên LMS của môn học. Sinh viên có thể nộp nhiều lần trước deadline; chỉ submission hợp lệ cuối cùng được chấm. Sinh viên có trách nhiệm nộp đủ sớm để xử lý các vấn đề compile hoặc nền tảng nếu có.

8 Các quy định khác

- Sinh viên phải tự hoàn thành bài tập lớn và bảo vệ source code của mình khỏi việc sao chép trái phép.
- Code đã nộp có thể được kiểm tra về mức độ tương đồng cấu trúc, các cấu trúc bị cấm và các pattern hiện thực bất thường. Một submission bị gắn cờ có thể được chọn để thực

hiện một phần kiểm tra ngăn về giải thích code hoặc chỉnh sửa code. Việc bị gắn cờ không tự động đồng nghĩa với bị xử phạt.

- Sinh viên có thể được yêu cầu giải thích hoặc thực hiện một thay đổi nhỏ trên bất kỳ phần nào trong implementation đã nộp.
- Hidden testcases không được công bố sau khi chấm. Giảng viên có thể thay vào đó cung cấp thông tin về nhóm testcase hoặc các dạng lỗi thường gặp.
- Mọi quyết định của giảng viên phụ trách bài tập lớn này là quyết định cuối cùng theo quy định của môn học và nhà trường.

9 Lịch sử thay đổi

—————HẾT—————